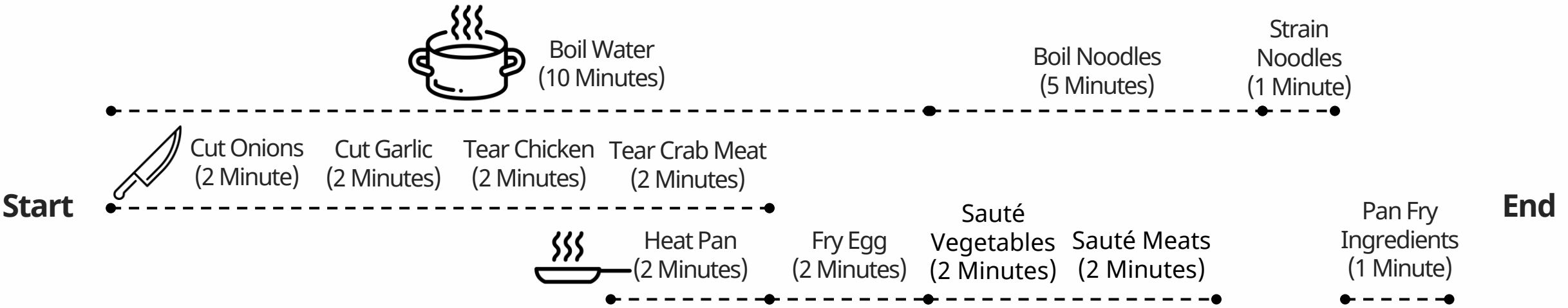
Graph-enhanced Large Language Models in Asynchronous Plan Reasoning

Lin, Fangru and La Malfa, Emanuele and Hofmann, Valentin and Yang, Elle Michelle and Cohn, Anthony G and Pierrehumbert, Janet B, ACML 2024

INDEX

01 Introduction

Research Background



1. Sequential: Doing everything one after another
2. Parallel: Doing everything at the same time
3. Asynchronous: Optimal Mixing of sequential and parallel tasks

01 Introduction

Research Significance

Can LLMs handle **asynchronous planning** tasks?

Planning is a fundamental aspect of human intelligence

1. Set of steps for a **task**
2. Calculate **time required for each step**
3. Set **step ordering constraints**

→ **Objective:**
Find the **shortest possible time**
while respecting **constraints**

Large Language Models have shown notions for the capability of this task

- Qualitive evidence but **no concrete quantitative evidence**
- First large-scale study investigating this question

01 Introduction

Research Significance

Most human tasks depend on asynchronous planning.

- Complex tasks can be impossible without asynchronous planning
- Practical Applications: Task automation, Project management, Development of autonomous robots

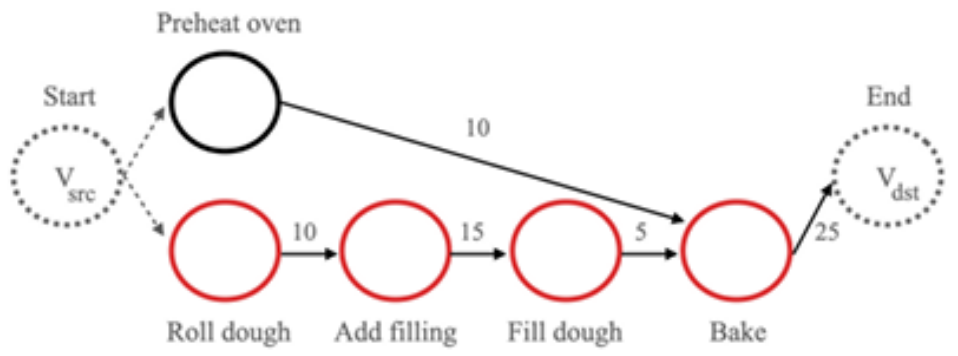
The paper seeks to provide contributions in…

- Create a new benchmark for asynchronous plan reasoning (AsyncHow)
- Develop a technique combining graphs with natural language prompts (PLaG)
- Comprehensive analysis of asynchronous task for SOTA LLM models
- Propose key limitations and capabilities of current LLMs in planning tasks

02 Technical Framework

Directed Acyclic Graphs

A Directed Acyclic** Graph is a conceptual representation of **series of activities**



Asynchronous Task

v_{src} : Start Node
v_{dst} : End Node

$$G(P) = \langle V, E, w \rangle$$

Where:

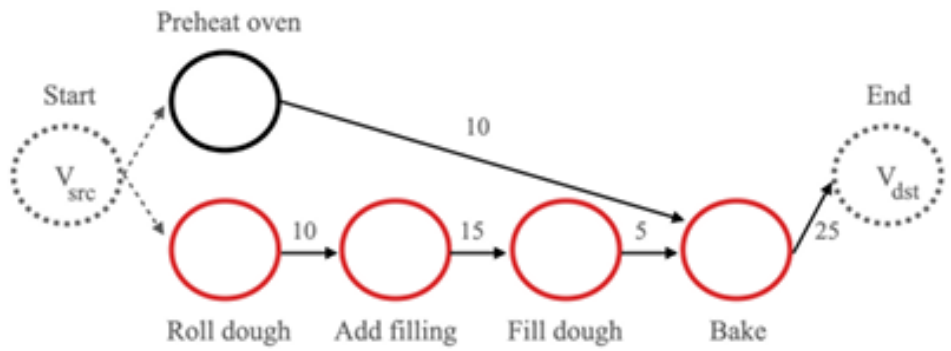
V : Set of nodes (steps in the plan)
E : Set of directed edges (ordering constraints)
w : Weight function (time durations)
P : The plan being represented

**Acyclic: There are no loops in the graph

02 Technical Framework

Mathematical Framework

Our goal is to find P*, the optimal plan that **minimizes the total time**.



Asynchronous Task

v_{src} : Start Node
v_{dst} : End Node

$$P^* \in \boldsymbol{arg\,min}\, TC(P)$$

$$\langle A, O, C \rangle$$

Where:

P^*: Optimal Path
$TC(P)$: Time cost
while respecting all constraints O and causal links C

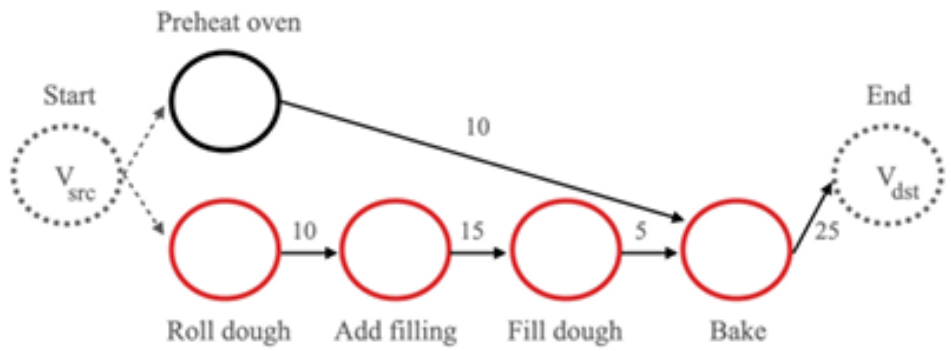
Additional Constraints:

1. Every node except our end node must have exactly **one successor** in the optimal path
2. Our START and END nodes must be part of the solution

02 Technical Framework

Mathematical Framework

Longest path in the graph is the **optimal plan time**



Asynchronous Task

v_{src} : Start Node
v_{dst} : End Node

The **minimum amount of time** needed for a plan would be the **longest sequence of dependent tasks**

1. Generate all valid subgraphs G' that meet the constraints
2. Calculate total path length for each G'

$$\sum(w(ei,j))\ for\ ei,j\ \in\ E'$$

3. Find G* with maximum path length

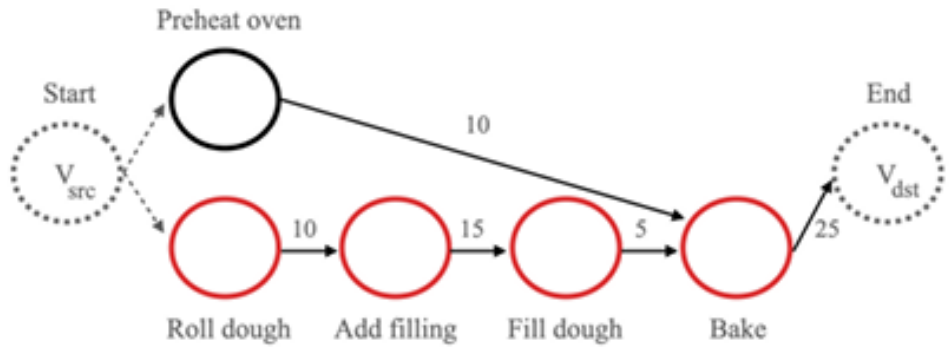
Deterministic Solution

Clear mathematical procedure that will always find the optimal solution

02 Technical Framework

Complexity Measure

Computation time **grows linearly** with the number of steps and dependencies



Asynchronous Task

v_{src} : Start Node
v_{dst} : End Node

For series-parallel graphs*, the optimal path has a linear complexity of

$$O(|V| + |E|)$$

Where:

|V| represents the number of steps
|E| represents the dependencies

1. Quickly find optimal plans even for large tasks
2. Adding new steps only increases computation time linearly
3. Provides a simple way to compare task difficulty"

* Series Parallel graphs are built by repeatedly combining smaller graphs either in series (one after another) or in parallel (side by side)
Eppstein, D. Parallel recognition of series-parallel graphs. Information and Computation, 98(1):41–55, 1992.

03 Dataset: AsyncHow

Data Generation

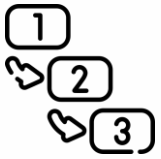
Create **natural planning scenarios** with time and dependency annotations

AsyncHow: Benchmark for asynchronous planning generated using an ***automatic*** method

- 1.6K high planning instances
- Sources: WikiHow, ProScript** (human-annotated)

Need to Extract

Time Estimations



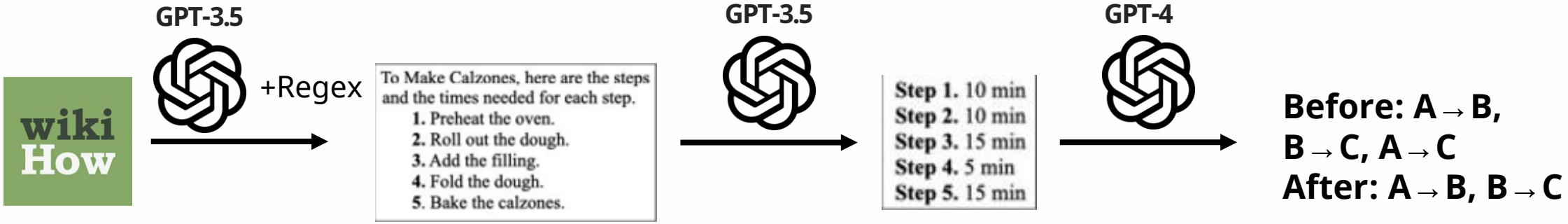
Dependencies

Optimal Solution

**ProScript is an end-to-end human annotated partial order plan dataset

03 Dataset: AsyncHow

Data Generation



Remove

- "WikiHows" with rating lower than 60%
- Optional steps
- Context-dependent instructions
- Unnecessary steps

Time Annotation

- 3 samples per step
- Kept longest estimation

Dependency Annotation

- 5 samples in DOT** language
- Required 4+ consistent answers

"Step 1 -> Step 2, Step 1 -> Step 3"

Can be expressed as

"Step 1 must precede Step 2, Step 1 must precede step 3, and succinctly as Step 1 must precede step 2 and 3"

Generate **natural language prompts** based on task information

GPT-4

Combine all asynchronous **instances** with **time annotation** for all steps in ProScript (1.6K Instances)

03 Dataset: AsyncHow

Data Generation

Human-annotated and **LLM-generated** instances receive **similar levels of acceptability**

"Consider the acceptability of the task time estimations and step ordering constraints."

	dev (in-domain)			test (cross-domain)		
	F1	P	R	F1	P	R
Humans	89.32	89.60	89.21	89.28	89.91	88.86
GPT-4	$89.80_{\pm 1.70}$	$90.65_{\pm 1.36}$	$89.30_{\pm 2.09}$	$85.59_{\pm 2.92}$	$85.95_{\pm 2.43}$	$85.77_{\pm 3.56}$

P: Pairwise precision. R: Recall, F1: F1 Score

Human Validation:
- 80 random samples
- Expert review
- Blind evaluation

≈

Automatic Validation:
- LLM annotated

03 Dataset: AsyncHow

Dataset Summary

Human-annotated and **LLM-generated** instances receive **similar levels of acceptability**

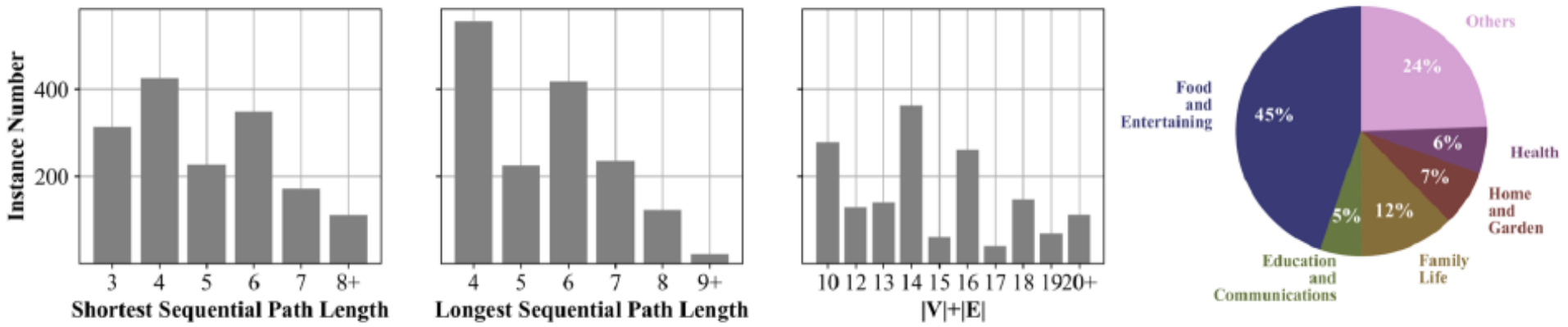


Figure 5. Overview of the AsyncHow benchmark. The three bar charts on the left display the instance numbers for the shortest/longest sequential path length and $|V| + |E|$ in different plans. The pie chart on the right shows the topic distribution in our dataset.

04 Methods

LLM Concepts

1. In-context Learning

- Prompt engineering method where demonstrations of the task are provided to the model as input

Prompt/Task

"Classify the following sentence based on sentiment"

Providing Examples (3-shot learning)

"Sentence: What a beautiful day.
Sentiment: Positive

Sentence: The weather is quite bad today.
Sentiment: Negative

Sentence: I have a desk in my living room.
Sentiment: Neutral"

Input

"Sentence: I'm so excited about the weekend.

Output

Positive

04 Methods

LLM Concepts

2. Chain of Thought (COT)

- Use series of intermediate reasoning steps to generate final output

Provide reasoning steps (1-shot COT) + Prompt/Task

"Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?"

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more how many apples do they have?

Output

A: The cafeteria had 23 apple originally. They used 20 to make lunch. So they had 23 – 20 = 3. They bought 6 more apples, so they have 3 + 6 = 9. The answer is 9.

Increases performances in complex tasks that require reasoning and planning

04 Methods

Baseline Approaches

1. Zero-shot Method: Prompting with only task description

Step Description

To create a video game, here are the steps and the times needed for each step.
Step 1. Learn the basics of programming (180 days)
Step 2. Learn to use a language that is used in games (60 days)
Step 3. Learn to use an existing game engine (30 days)
Step 4. Program the game (90 days)
Step 5. Test the game (30 days)

Step Dependencies

These ordering constrains need to be obeyed when executing above steps:
Step 1 must precede step 2.
Step 1 must precede step 3.
Step 2 must precede step 4.
Step 3 must precede step 4.
Step 4 must precede step 5.

Task

Question: Assume that you need to execute all the steps to complete the task and that infinite resources are available. What is the shortest possible time to create a video game? Answer the time in double quotes.
Answer:

04 Methods

Baseline Approaches

2. Zero-shot + CoT Method: Prompting with task description and instruction

Step Description

Step Dependencies

Task + **Reasoning**

To create a video game, here are the steps and the times needed for each step.
Step 1. Learn the basics of programming (180 days)
Step 2. Learn to use a language that is used in games (60 days)
Step 3. Learn to use an existing game engine (30 days)
Step 4. Program the game (90 days)
Step 5. Test the game (30 days)

These ordering constrains need to be obeyed when executing above steps:
Step 1 must precede step 2.
Step 1 must precede step 3.
Step 2 must precede step 4.
Step 3 must precede step 4.
Step 4 must precede step 5.

Question: Assume that you need to execute all the steps to complete the task and that infinite resources are available. Let's think step by step and then answer the time in double quotes.
Answer:

04 Methods

Baseline Approaches

3. K-shot Method: Prompting with *k* in-context instances with desired outputs

Step Description

Step Dependencies

Task + **Desired Output**

To create a video game, here are the steps and the times needed for each step.
....
Step 5. Test the game (30 days)

These ordering constrains need to be obeyed when executing above steps:
....
Step 4 must precede step 5.

Question: Assume that you need to execute all the steps to complete the task and that infinite resources are available. What is the shortest possible time to Make Calzones? Answer the time in double quotes.
Answer: The shortest possible time to Make Calzones is "55 min".
...[TWO MORE EXAMPLES]...
###
[ZERO SHOT PROMPT]

04 Methods

Baseline Approaches

4. K-shot + CoT Method: Provide reasoning steps with desired output

Step Description + Dependencies

Task + **Reasoning Steps** + **Desired Output**

To create a video game, here are the steps and the times needed for each step.
....
Step 4 must precede step 5.

Question: Assume that you need to execute all the steps to complete the task and that infinite resources are available. What is the shortest possible time to Make Calzones? Answer the time in double quotes.
Answer: Since step 1 must precede step 5, step 2 must precede step 3, step 3 must precede step 4, step 4 must precede step 5, we can conclude that we must execute step 2, step 3, step 4, then step 5 sequentially, and since step 1 happens before step 5,it can be done in parallel with step 2, 3, and 4, preceding step 5. Since sequentially executing step 2, 3, 4, and 5 takes 10 + 15 + 5 + 25 = 55 min, while sequentially executing step 1 then step 5 only takes 10 + 25 = 35 min, the shortest possible time to Make Calzones is "55 min".
...[TWO MORE EXAMPLES]...
###
[ZERO SHOT+COT PROMPT]

04 Methods

Plan Like a Graph Structure

5. Plan Like a Graph Method: Represent planning problems into graphs

Step Description + Dependencies

###Examples:
To Make Calzones, here are the steps and the times needed for each step.
Step 1. Preheat the oven to 425 degrees. (10 min)
Step 2. Roll out the dough. (10 min)
Step 3. Add the filling. (15 min)
Step 4. Fold and pinch the dough. (5 min)
Step 5. Bake the calzones. (25 min)

These ordering constraints need to be obeyed when executing above steps:
Step 1 must precede step 5.
Step 2 must precede step 3.
Step 3 must precede step 4.
Step 4 must precede step 5.

04 Methods

Plan Like a Graph Structure

5. Plan Like a Graph Method: Represent planning problems into graphs

Step Description + Dependencies

Representation as graph

###Examples:
To Make Calzones, here are the steps and the times needed for each step.
...
Step 5. Bake the calzones. (25 min)

These ordering constraints need to be obeyed when executing above steps:
...
Step 4 must precede step 5.

Here is the adjacency list representation of the step ordering constraints:
{'1': ['5'], '2': ['3'], '3': ['4'], '4': ['5'], '5': ['END'], 'END': [], 'START': ['1', '2']}
Time for each step can be represented as a dictionary:
{'1': '10 min', '2': '10 min', '3': '15 min', '4': '5 min', '5': '25 min'}

04 Methods

Plan Like a Graph Structure

5. Plan Like a Graph Method: Represent planning problems into graphs

Step Description + Dependencies

Representation as graph

CoT Reasoning + K-shot

###Examples:
To Make Calzones, here are the steps and the times needed for each step.
...
Step 4 must precede step 5.

Here is the adjacency list representation of the step ordering constraints:
...
{'1': '10 min', '2': '10 min', '3': '15 min', '4': '5 min', '5': '25 min'}

***Question**: Assume that you need to execute all the steps to complete the task and that infinite resources are available. What is the shortest possible time to Make Calzones? Answer the time in double quotes.*
***Answer**: Since step 1 must precede step 5, step 2 must precede step 3, step 3 must precede step 4, step 4 must precede step 5, we can conclude that we must execute step 2, step 3, step 4, then step 5 sequentially, and since step 1 happens before step 5, it can be done in parallel with step 2, 3, and 4, preceding step 5. Since sequentially executing step 2, 3, 4, and 5 takes 10 + 15 + 5 + 25 = 55 min, while sequentially executing step 1 then step 5 only takes 10 + 25 = 35 min, the shortest possible time to Make Calzones is "55 min".*

04 Methods

Plan Like a Graph Structure

5. Plan Like a Graph Method: Represent planning problems into graphs

Step Description + Dependencies

Representation as graph

CoT Reasoning + K-shot

Task

###Examples:
To Make Calzones, here are the steps and the times needed for each step.
...
Step 4 must precede step 5.

Here is the adjacency list representation of the step ordering constraints:
...
{'1': '10 min', '2': '10 min', '3': '15 min', '4': '5 min', '5': '25 min'}

***Question**: Assume that you need to execute all the steps to complete the task and that infinite resources*
...
Calzones is "55 min".

Question: Assume that you need to execute all the steps to complete the task and that infinite resources are available. What is the shortest possible time to create a video game? Let's think step by step and then answer the time in double quotes.
Answer:

04 Methods

Plan Like a Graph Structure

5. Plan Like a Graph Method: Represent planning problems into graphs

Two variations of PLaG

1. PLaG (Explicit graph): Provide explicit graphs in prompt

   - When asking to find the shortest path, explicitly **provide the adjacency list of the graph** being questioned

2. PLaG (Build a Graph): Let the LLM build their own graph

   - Instead of providing the adjacency list of the graph in question, **ask the LLM to devise their own graph**.

###Task in Question
...
Question: Assume that you need to execute all the steps to complete the task and that infinite resources are available. What is the shortest possible time to create a video game? Let's construct a graph with the nodes and edges first to represent step ordering constraints, and also construct a dictionary to represent time needed for each step. Use the graph and dictionary to calculate the shortest possible time needed for the task. Let's think step by step and then answer the time in double quotes.
Answer:

05 Results

Experiment

Evaluation

- Calculate **accuracy of correctly reported optimal path (P*)** needed for each asynchronous task based on the AsyncHow benchmark

Model Variants

- 3 Closed-source LLMs: GPT-3.5, GPT-4, Command
- 2 Open-source LLMs: LLaMA-2-70B-chat, Mistral-7B-Instruct
- 3-shot used for in-context learning (3 examples)

05 Results

Findings

PLaG **boosts the planning accuracy** of asynchronous tasks **across all model**

	Without PLaG				With PLaG	
Model	zero-shot	zero-shot + CoT	*k*-shot	*k*-shot + CoT	PLaG (explicit graph)	PLaG (BaG)
GPT-4	0.130	0.129	0.107	0.657	0.730†	**0.777**†
GPT-3.5	0.199	0.224	0.248	0.226	0.290†	**0.355**†
Command	0.078	0.015	0.050	0.078	**0.100**	0.050
LLaMA-2-70B-chat	0.039	0.038	0.053	0.076	**0.101**†	0.069
Mistral-7B-Instruct	0.078	0.070	0.098	0.149	**0.161**	0.146

Denote with † when the performances with PLaG are significantly better ($p < 0.05$) than the best result without.

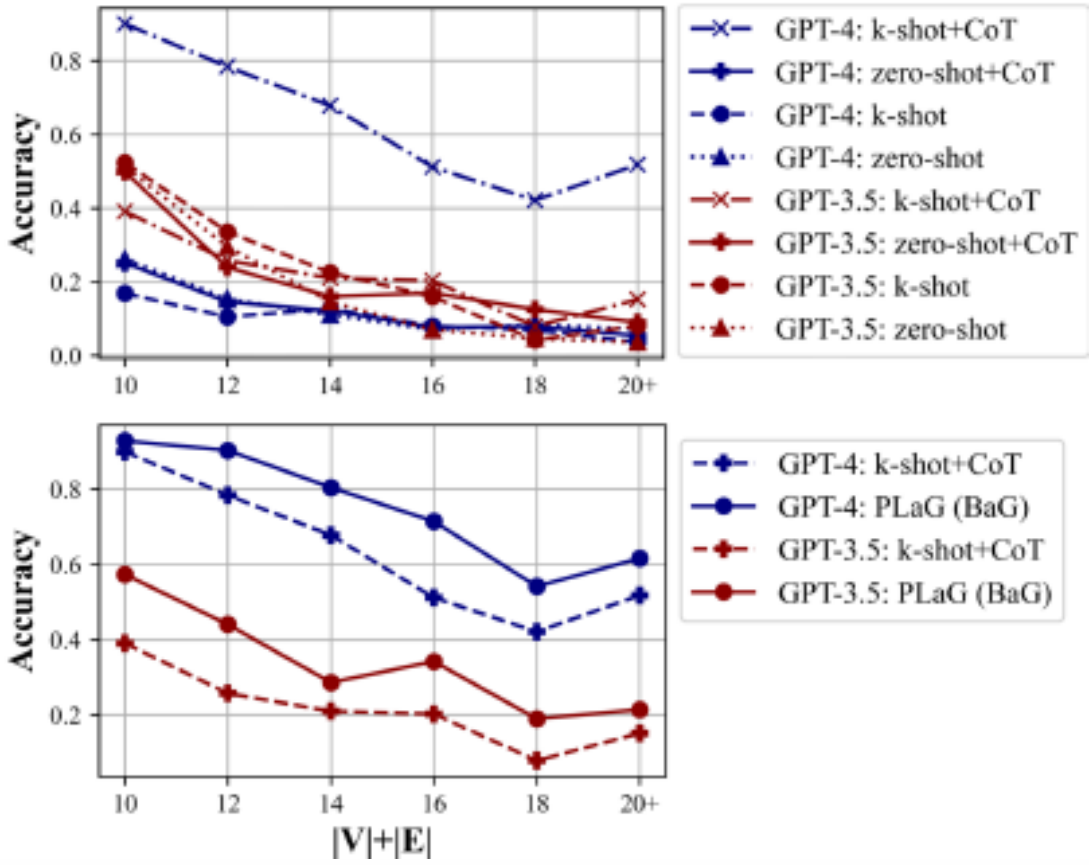
- GPT-4 jumped from 65.7% (best traditional method) to 77.7% with PLaG
- All models perform poorly in zero-shot settings and can be improved through PLaG implementation
- CoT prompting is essential in planning task
- Letting the LLMs build their own graph enhance their ability to understand the task

05 Results

Findings

Model **accuracy** consistently **decreases** as task **complexity increases**

- Performance of PLaG –based models consistently outperform other methods across all complexity levels
- The performance of all models struggle significantly when the complexity exceeds 18
- Performance seems to slightly improve when complexity exceeds 20**



**Plans with complexity ≥ 20 had simpler time unit conversions (Sec vs Min, Hour vs Hour)

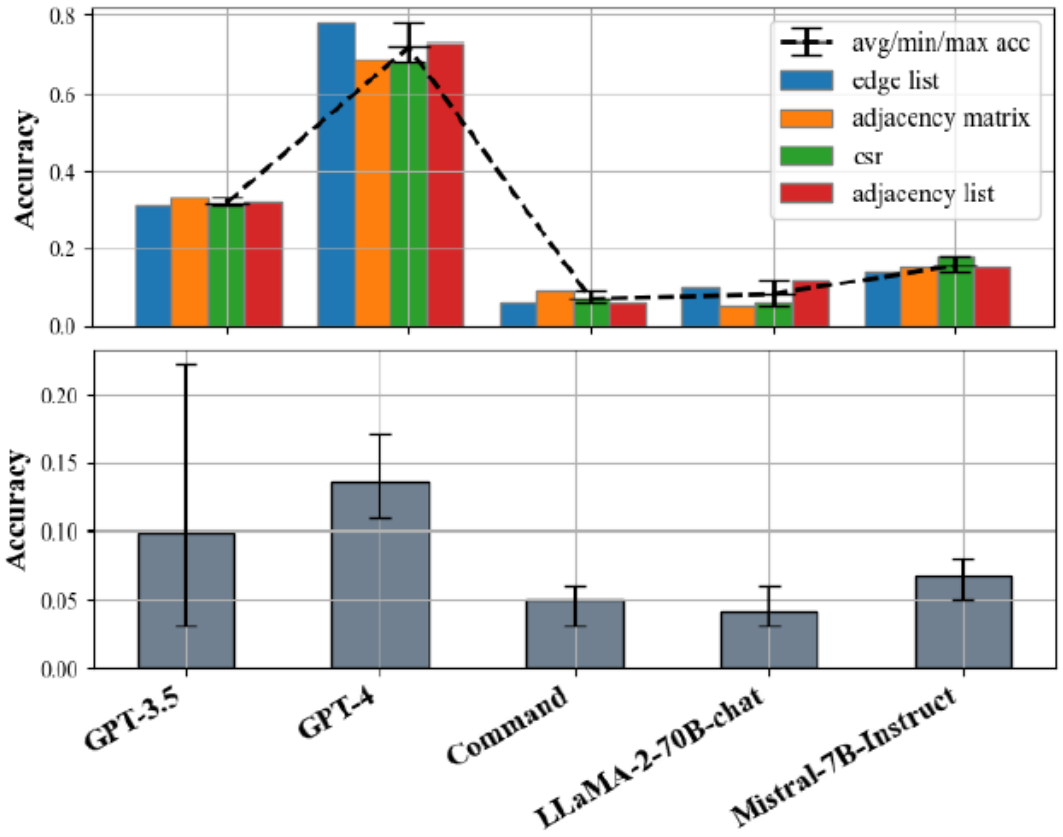
05 Results

Ablation Studies

Does the **type of graph** affect the performance?

- Type of graph components

  Edge list, Adjacency Matrix, CSR, Adjacency List
- 100 instances per different graph types were tested
- LLMs differ in graph types they prefer



05 Results

Ablation Studies

Why PLaG shows **better performance**? – from out-of-distribution experiment

- As most tasks have a complexity lower than 20, the authors made a synthetic dataset
  - 2000 examples of tasks with complexity between 10 and 40
  - Synthetic dataset consists of dynamic programming tasks without real-life natural language context
- Model performance was better in synthetic dataset
- This can be the reason of better performance of PLaG: **PLaG change the natural language question into more familiar form, graph.**

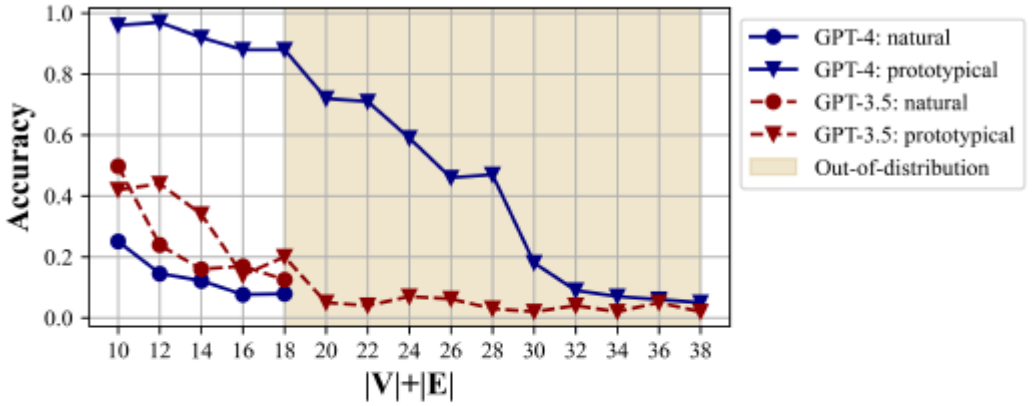


Figure 8. GPT-3.5 and GPT-4 performance on *prototypical* longest path search problem and on *natural* AsyncHow task with zero-shot + CoT. Models' respective best graph types found in Section 4.2 are used in prototypical probing.

06 Discussions

Why does **PLaG (BaG) performs better** than PLaG (Explicit graph)?

- Difference in prompt positioning causes differences in performance

  PLaG (Explicit Graph) format: [Task description with **graph**] Answer:"

  BaG Format: "[Task description] Answer: **[Graph]**

- The location of the graph causes the LLM to overlook the graph
- Active vs. Passive Processing

  Active construction of the graph enables deeper problem understanding for the BaG method

07 Research Ideas

Complex Planning Problem

Performance drops substantially when complexity exceeds 20

- Much of the tasks in the real world are much more complicated than we realize ($|V| + |E| > 20$)
  - Building a software, Building a car, Preparing a paper presentation
- Real-world scenarios suffer from a limited resource constraint

07 Research Ideas

Compositional Planning

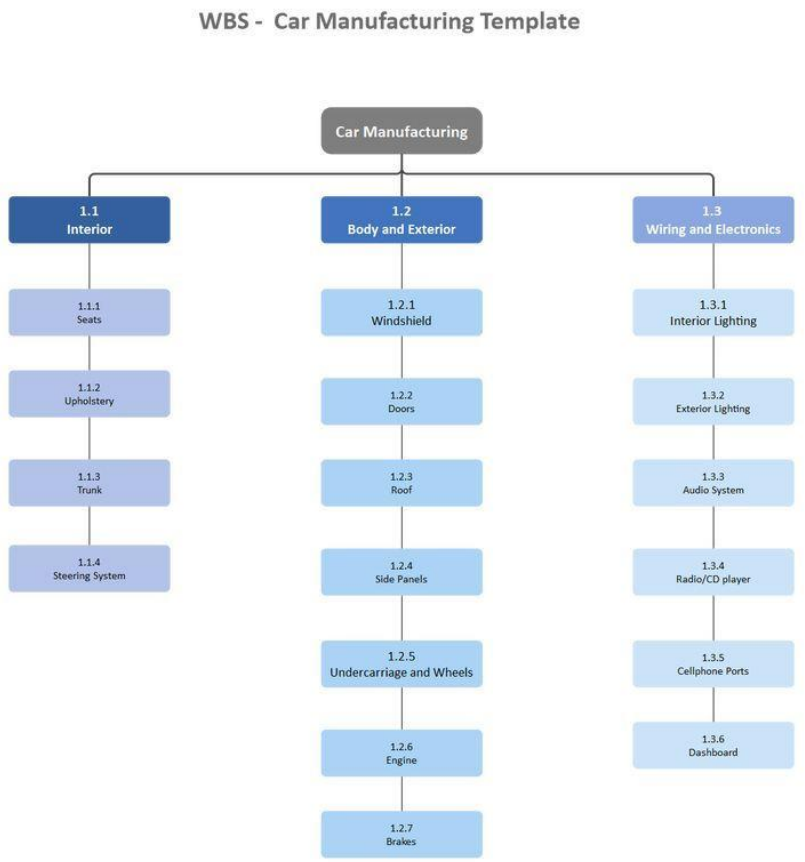
Decomposing the complex process into sub tasks

- Car Assembly process example

  For someone to assemble a car, there are 30,000 to 40,000 parts and requires the cooperation of multiple teams.

- Can be broken down into

  Chassis & Frame Phase, Frame assembly, Suspension system, Wheel mounting, Electrical systems, Interior & Finishing…



07 Research Ideas

Enhanced PLaG

Distinguishing between higher-order and lower order dependencies and steps

- Hierarchical Planning
  - Phase Dependencies
  - Resource Management
- Component Flow Tracking
- Quality Checkpoints within DAG

07 Research Ideas

Different Prompting Mechanisms

Are graphs the best way for prompting LLMs?

- This graph-prompting method had better performance than normal CoT.
- Using symbols to plan a task ->
- LLMs may be better at understanding pseudo-code, visual information, tables, etc.

NLVR-based Manipulation

Story: There are three boxes. In the left box, there are one large round in black, one small square in blue, one small square in blue, one small triangle in yellow, one middle square in yellow. In the middle box, there are one large square in yellow, one middle triangle in blue, one large round in black. In the right box, there are one large square in blue, one large triangle in black, one middle triangle in black.
Question: How to move all black objects to the left box?
Answer:
To move all black objects to the left box, we need to identify all the black objects in the three boxes.
They are:
- left box: **(large, round, black)**
- middle box: **(large, round, black),**
- right box: **(middle, triangle, black), (large, triangle, black)**

Then we can move the objects not in the left box above to the left box:
1. **(large, round, black):** middle - left
2. **(middle, triangle, black):** right - left
3. **(large, triangle, black):** right - left

Chain-of-Symbol Prompting Elicits Planning in Large Langauge Models (Hu et al. 2023)

07 Research Ideas

Enhanced PLaG

Distinguishing between higher-order and lower order dependencies and steps

- Hierarchical Planning
  - Phase Dependencies
  - Resource Management
- Component Flow Tracking
- Quality Checkpoints within DAG

Q&A

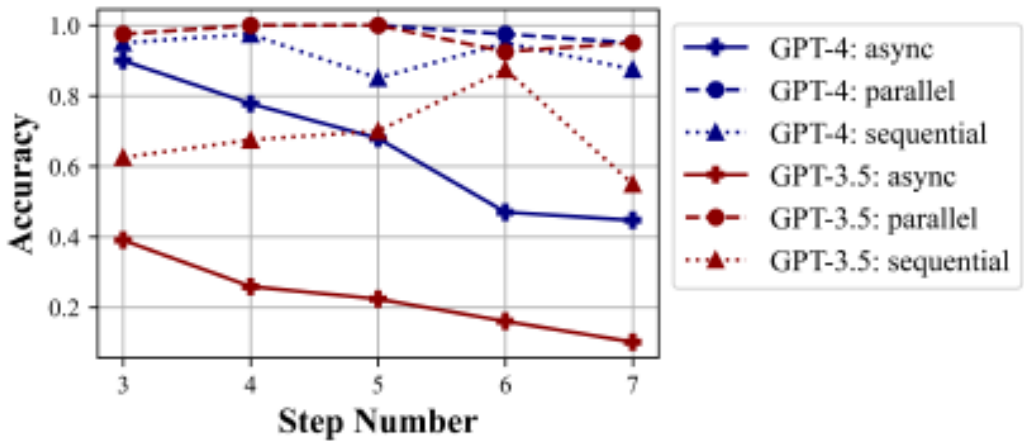
08 Appendix

Ablation Studies

Does the **type of task** or the affect the performance?

- Type of task components

  Sequential, Parallel, Asynchronous
- 200 instances per different task type (3-7 tasks)
- K-shot + CoT setting (best traditional method)
- GPT 3.5 and GPT 4 performed similar in parallel tasks
- GPT 4 performs better in sequential tasks
- Both models underperform in asynchronous tasks



08 Appendix

Ablation Studies

Prototypical task (edge list)

The following lists of nodes [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10] and edges [[0, 1, 1], [1, 2, 1],[1, 3, 1], [2, 10, 1], ..., [9, 10, 5]] define a directed acyclic graph. Each element in the list of edges is expressed in the form (i,j,w), and specifies that node i connects to node j with weight w. What is the length of the longest path from node 0 to node 10? Think step by step and then reply with the numerical value of the shortest path enclosed by <result><result> tags.
Answer: